

# Exploring Hallucination Detection in Large Language Models

Ajay Rajendra Kumar, Vicky Daniel, Vaibhav Sankaran

CS6120 Natural Language Processing - Team 111

## Abstract

Hallucination in large language models (LLMs) refers to the generation of text that is fluent and coherent but factually incorrect. This is a critical problem in open-domain question answering, where users rely on model outputs for accurate information. In this work, we build upon MIND, an unsupervised framework for real-time hallucination detection that leverages the internal states of LLMs. We reproduce the original MIND pipeline and extend it with four alternative feature extraction strategies: multi-layer last-token concatenation (Method 1), sparse-layer mean pooling (Method 2), inter-layer delta features (Method 3), and a multi-signal ensemble (Method 4). We evaluate these strategies on two models, GPT-J-7B and Llama-3-8B, using a self-supervised data generation pipeline and a newly constructed HELM benchmark. Results show that the aggregation function is the dominant factor: mean pooling (multi\_layer\_mean) is the most robust method for Llama-3-8B, while the original MIND configuration is strong and performs the best for GPT-J-7B. Delta-based and last-token representations are consistently weak across different model architectures.

## I. Introduction

Large language models such as GPT-J and Llama-3 have demonstrated strong capabilities across a wide range of text generation tasks. However, a well-known failure mode is hallucination, where the model produces output that sounds plausible but is factually incorrect or unsupported by any reliable source [1]. This undermines trust in LLMs, particularly in high-stakes applications such as open-domain question answering, medical information retrieval, and legal analysis.

Existing approaches to hallucination detection fall into three broad categories. Sampling-based methods such as SelfCheckGPT [2] generate multiple responses to the same prompt and measure consistency. These methods require  $N$  forward passes per query, making them computationally prohibitive at inference time. Retrieval-augmented generation (RAG) [4] grounds outputs in retrieved documents but prevents rather than detects hallucinations. Supervised probing methods, such as ITI [3], use hidden representations to predict truthfulness but require expensive labeled datasets.

This work builds upon MIND [1], an unsupervised framework that addresses these limitations by leveraging the model's own internal states during inference. MIND generates pseudo-labeled training data from Wikipedia in a self-supervised manner and trains a lightweight MLP to detect hallucinations in real time at the sentence level.

Our contributions are as follows:

1. We reproduce the original MIND pipeline so as to experiment with our strategies over it.

2. We design and evaluate four alternative feature extraction strategies exploring different ways of aggregating internal LLM representations.
3. We extend the HELM benchmark for GPT-J-7B and Llama-3-8B using a self-supervised pipeline and an LLM-as-judge evaluation paradigm.
4. We analyze how aggregation strategies affect detection performance across architectures.

## II. Prior Work

Hallucination detection has received growing attention as LLMs are deployed in factual reasoning tasks.

### A. Sampling-Based Detection

SelfCheckGPT [2] generates multiple responses to the same query and measures consistency across samples as a proxy for factuality. While effective, this requires  $N$  forward passes per input, making it slow and expensive for real-time use.

### B. Retrieval-Augmented Generation

RAG [4] reduces hallucination by grounding outputs in retrieved documents. However, it is a prevention strategy: if the model hallucinates despite retrieval, the error goes undetected.

### C. Internal State Probing

SAPLMA [5] showed that hidden states at the final layer are predictive of truthfulness when trained with supervised labels. Inference-Time Intervention (ITI) [3] directly modifies attention head activations at test time. Both methods require labeled, model-specific training data. MIND [1] eliminates this requirement through Wikipedia-based self-supervision.

## III. Base Paper: The MIND Framework

Our work is based on "Unsupervised Real-Time Hallucination Detection based on the Internal States of Large Language Models" (Su et al., ACL Findings 2024) [1]. MIND is an unsupervised, real-time, and model-agnostic framework for hallucination detection.

The framework proposes multiple strategies to obtain a unified representation of the internal states. Among these, the best performing approach extracts the last-token embedding for each generated token when an input sequence is processed by a frozen LLM. These per-layer last-token embeddings are then averaged across the sequence dimension using a mean operator to produce a unified feature representation.

Additionally, embeddings of all the tokens from the final layer are averaged along the sequence dimension and concatenated with the previously obtained feature representation. Finally, a lightweight MLP, trained on this

concatenated embedding, performs sentence-level prediction to determine whether the generated output is hallucinated.

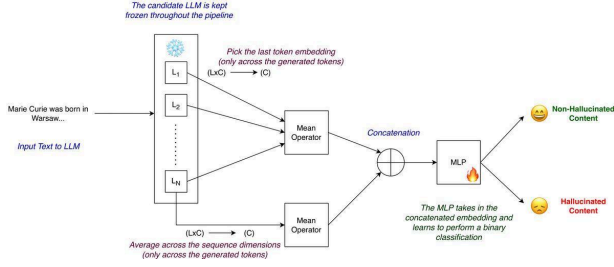


Fig. 1. MIND system architecture (Su et al., 2024).

Training data is generated self-supervisedly using the WikiText-103 dataset [6]. In this process, named entities in the model's continuation are extracted using spaCy and compared against ground-truth entities from the original passage. Matches yield non-hallucinated labels; mismatches yield hallucinated labels.

To support reproducibility, the authors introduced HELM (Hallucination Evaluation for Multiple LLMs), a benchmark covering six LLMs with human-annotated sentence-level labels and pre-computed internal states. They also share the pipeline using which this benchmark was created, so that it could be extended for other LLMs as well.

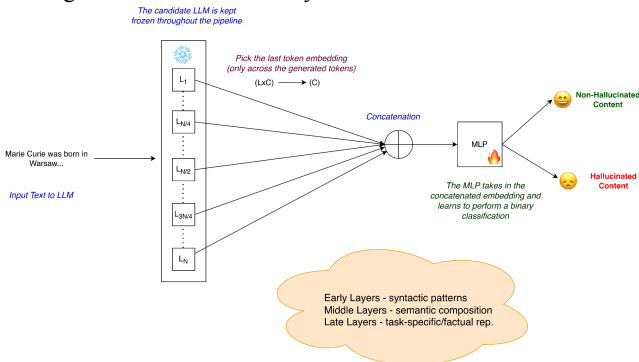
#### IV. Methods

We propose four alternative feature extraction strategies. In all methods the candidate LLM is frozen throughout the pipeline.

##### A. Method 1: Multi-Layer Last-Token Concatenation

Method 1 samples layers at evenly spaced intervals across model depth (layers 1,  $N/4$ ,  $N/2$ ,  $3N/4$ ,  $N$ ). From each sampled layer the last-token embedding across generated tokens is extracted, yielding a vector of dimension  $C$  per layer. These vectors are concatenated and passed to the MLP. The motivation is that early layers encode syntactic patterns, middle layers encode semantics, and late layers encode task-specific and factual representations. The schematic of this method is shown below:

Fig. 2. Method 1: Multi-Layer Last-Token Concatenation

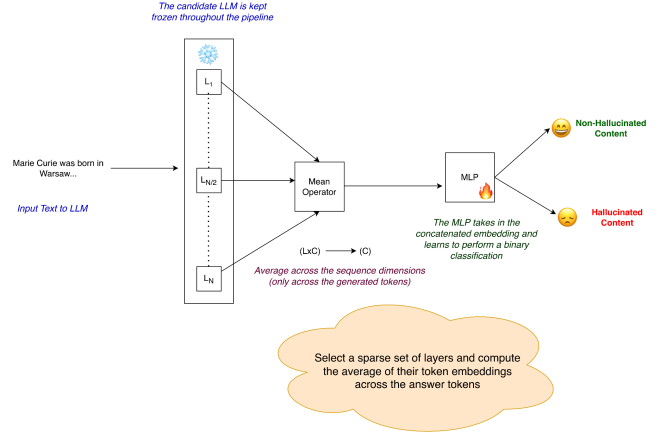


##### B. Method 2: Sparse-Layer Mean Pooling

Method 2 selects a sparse set of layers and, rather than taking the last token, computes the mean of token

embeddings across the entire answer span for each selected layer (generated tokens only). This per-layer mean vector is fed directly into the MLP. The intuition is that averaging across generated tokens provides a more stable summary than a single endpoint token. The schematic of this method is shown below:

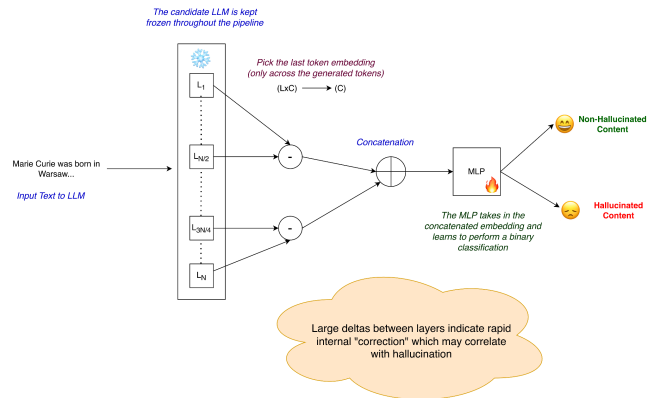
Fig. 3. Method 2: Sparse-Layer Mean Pooling



##### C. Method 3: Inter-Layer Delta Features

Method 3 computes the element-wise difference between embeddings at consecutive sampled layers. For each adjacent pair  $(L_k, L_{k+1})$ , a delta vector is produced. These deltas are concatenated and fed into the MLP. We hypothesized that large numerical deltas may indicate rapid internal correction during the forward pass, which may correlate with hallucination. The schematic of this method is shown below:

Fig. 4. Method 3: Inter-Layer Delta Features



##### D. Method 4: Multi-Signal Ensemble

Method 4 is a kitchen-sink method that concatenates the feature vectors from Methods 1, 2, and 3 before passing the combined representation to the MLP, allowing the classifier to learn which signals are most informative for each target model.

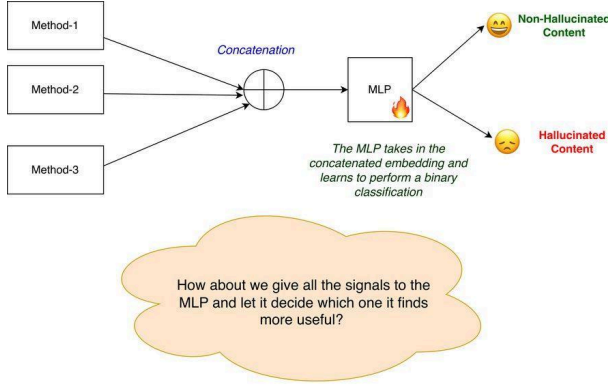


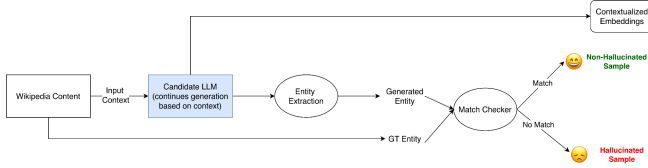
Fig. 5. Method 4: Multi-signal ensemble.

## V. Dataset

### A. Self-Supervised Training Pipeline

Model-specific training datasets are constructed using a self-supervised pipeline adapted from MIND. WikiText-103 is used as input context. The candidate LLM generates continuations while contextualized embeddings are extracted. Named entities are identified using spaCy and compared against Wikipedia ground-truth. Matches label sentences as non-hallucinated; mismatches label them as hallucinated. A schematic of this pipeline is demonstrated in the figure below:

Fig. 6. Self-supervised training data generation pipeline



### B. HELM Benchmark Generation

We extend the HELM benchmark to evaluate GPT-J-7B and Llama-3-8B, incorporating newer models and representation strategies that are not part of the original HELM setup. The original HELM benchmark relies on human annotators for ground truth labeling. However, due to limited availability of human annotation resources in this project, we adopt the LLM-as-judge paradigm.

Although this may appear paradoxical, since LLMs are used to detect hallucinations in other LLMs, we attempt to mitigate this concern by employing a strong commercial foundation model, GPT-4.1-mini, as the evaluator. Given its extensive world knowledge and strong instruction-following capabilities, we design prompts that encourage the model to ground its judgments in factual knowledge before making a decision. This allows us to treat its outputs as reliable pseudo ground truth labels for the purpose of extending the HELM benchmark within the scope of this project. The system prompt used for this evaluation is provided in the appendix.

We use WikiText-103 as the source of input context. The target LLM generates continuations, during which contextualized embeddings are extracted and stored. The judge model then evaluates the generated text for factual

consistency with the corresponding Wikipedia source. Outputs that are consistent with the source are labeled as non hallucinated, while inconsistent outputs are labeled as hallucinated.

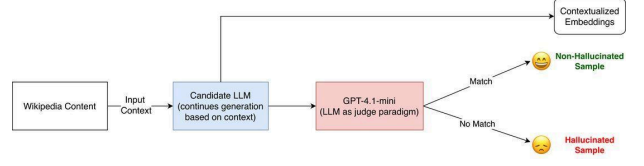


Fig. 7. HELM benchmark generation pipeline using LLM-as-judge.

Dataset statistics are shown in Table I. Both models use 6,000 training samples, 1,000 validation samples, and 1,304 test samples. More details about the data can be found in the table below:

| Model        | Split | Non Hallucinated | Hallucinated | Total |
|--------------|-------|------------------|--------------|-------|
| llama3base8b | train | 6000             | 2061         | 8061  |
|              | valid | 1000             | 362          | 1362  |
|              | test  | 1304             | 430          | 1734  |
| gptj7b       | train | 6000             | 3709         | 9709  |
|              | valid | 1000             | 657          | 1657  |
|              | test  | 1304             | 795          | 2099  |

TABLE I. Dataset split counts for each model.

## VI. Results

We evaluate all methods using four metrics defined by the HELM benchmark: Sentence-Level AUC, Passage-Level AUC, Sentence-Level Correlation, and Passage-Level Correlation. AUC is threshold-independent and measures classification performance, while correlation captures the alignment between classifier confidence and ground truth labels.

HELM considers hallucination detection at two granularities: sentence level and passage level. Since each annotated sentence is part of a passage, the benchmark also evaluates passage-level hallucination detection. A passage is classified as hallucinated if any of its constituent sentences is identified as hallucinated.

All metrics are reported such that higher values indicate better performance. Results are presented in Tables II and III, with bold values denoting the best result within each model column.

| Strategy     | S-AUC GPT-J  | S-AUCL Llama-3 | P-AUC GPT-J  | P-AUCL Llama-3 |
|--------------|--------------|----------------|--------------|----------------|
| MIND (orig.) | <b>88.17</b> | 53.52          | <b>96.69</b> | 90.81          |
| ml_mean      | 72.60        | <b>54.39</b>   | 97.93        | <b>93.17</b>   |
| ml_concat    | 69.87        | 52.32          | 94.93        | 92.08          |
| ml_deltas    | 68.02        | 51.18          | 94.31        | 90.53          |

|             |       |       |       |       |
|-------------|-------|-------|-------|-------|
| ml_last_tok | 68.84 | 50.49 | 94.05 | 90.53 |
|-------------|-------|-------|-------|-------|

TABLE II. AUC at sentence (S) and passage (P) level.

| Strategy     | S-CorrG<br>PT-J | S-CorrL<br>lama-3 | P-CorrG<br>PT-J | P-CorrL<br>lama-3 |
|--------------|-----------------|-------------------|-----------------|-------------------|
| MIND (orig.) | <b>0.46</b>     | 0.16              | <b>0.51</b>     | 0.29              |
| ml_mean      | 0.23            | <b>0.20</b>       | 0.31            | <b>0.29</b>       |
| ml_concat    | 0.16            | 0.14              | 0.11            | 0.21              |
| ml_deltas    | 0.15            | 0.11              | 0.06            | 0.13              |
| ml_last_tok  | 0.13            | 0.11              | 0.03            | 0.10              |

TABLE III. Correlation at sentence (S) and passage (P) level.

### A. Model-Dependent Performance

Results reveal strong model-dependency. MIND (original) achieves the highest sentence-level AUC on GPT-J-7B (88.17) and sentence-level correlation (0.46), but performs near chance-level on Llama-3-8B with a sentence AUC of 53.52 and correlation of 0.16. This demonstrates that MIND, while strong on its target model, fails to generalize across architectures.

In contrast, multi\_layer\_mean is competitive or superior on Llama-3-8B across all four metrics, achieving the highest sentence-level AUC (54.39), passage-level AUC (93.17), and highest correlation at both levels (0.20 and 0.29). It is the only method that consistently ranks among the top two performers for Llama-3-8B.

### B. Impact of Aggregation Strategy

Mean pooling (multi\_layer\_mean) significantly outperforms naive concatenation (multi\_layer): on Llama-3-8B, sentence AUC improves from 52.32 to 54.39 and passage AUC from 92.08 to 93.17. This demonstrates that how representations are combined matters more than the number of representations included.

Delta-based features (ml\_deltas) and last-token representations (ml\_last\_tok) are consistently among the weakest strategies across both models and all four metrics, suggesting that inter-layer change signals and isolated endpoint embeddings are not reliable hallucination indicators at the scales evaluated.

## VII. Discussion

### A. Scientific Findings

The most significant finding is that the aggregation function has a larger impact on performance than the choice of which layers to sample. Mean pooling produces a more stable and representative summary of the model's internal state than concatenation or last-token extraction, consistent with the broader NLP literature on pooling strategies for sentence representation.

The failure of MIND (original) on Llama-3-8B highlights that internal representations are architecture-specific. A detector tuned on GPT-J measures patterns that do not have analogues in Llama-3, reinforcing the need for model-adaptive approaches. The delta-based method did not

yield improvements despite its motivation; the layer-to-layer changes computed may be too noisy at the scales evaluated.

### B. Challenges

Three primary challenges were encountered as part of implementation, testing and evaluation phases of the project. First, data scarcity: the original HELM benchmark does not include data for newer models like Llama-3-8B, requiring full benchmark generation from scratch for each new model added, resulting in increased compute requirements. Furthermore, GPT-4.1-mini API calls for the LLM-as-judge evaluation and introduces budget limitations. Second, resource constraints: large-scale inference required significant GPU compute, which was not available with every one of us. Third, implementation challenge: integrating the custom pipeline with the original MIND codebase required extensive refactoring to support different model interfaces and internal state extraction routines.

### C. Strengths and Shortcomings

A key strength is the reproducibility of the data generation pipeline: adapting MIND's self-supervised approach to new architectures shows the framework's generalizability. The LLM-as-judge paradigm for HELM generation also provides a scalable alternative to human annotation.

The primary shortcoming is limited model coverage. Evaluation on only two models makes it difficult to draw broad conclusions. The small test set (1,304 samples) means differences between closely performing strategies may not be statistically reliable. Method 4 did not improve over multi\_layer\_mean, potentially because the increased input dimensionality overwhelmed the classifier given the available training data.

### C. Future Work

Future work can explore both architectural and model-level extensions. On the architectural side, the current lightweight MLP can be replaced or augmented with more expressive classifiers, such as deeper MLPs, attention-based pooling mechanisms, or transformer-style heads that better capture inter-layer and inter-token dependencies in the representation space. On the model side, the evaluation can be expanded to include a broader spectrum of foundation models, spanning different scales, training paradigms, and architectural families, in order to assess the generalizability and robustness of the proposed hallucination detection framework across diverse LLM behaviors.

## VIII. Individual Contributions

**Ajay Rajendra Kumar:** Method 1, Method 4, literature review, experiment design, evaluation methodology.

**Vicky Daniel:** Method 2, self-supervised data generation pipeline, codebase development and debugging.

**Vaibhav Sankaran:** Method 3, HELM benchmark generation pipeline, results analysis and interpretation, report and presentation preparation.

## IX. References

- [1] W. Su et al., "Unsupervised Real-Time Hallucination Detection based on the Internal States of Large Language Models," Findings of ACL 2024, pp. 14379–14391.
- [2] P. Manakul, A. Liusie, and M. J. F. Gales, "SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models," EMNLP 2023.
- [3] K. Li et al., "Inference-Time Intervention: Eliciting Truthful Answers from a Language Model," NeurIPS 2023.
- [4] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," NeurIPS 2020.
- [5] A. Azaria and T. Mitchell, "The Internal State of an LLM Knows When It's Lying," Findings of EMNLP 2023.
- [6] Su, Weihang, et al. "Unsupervised real-time hallucination detection based on the internal states of large language models." Findings of the Association for Computational Linguistics: ACL 2024. 2024.

## A. Appendix: User Manual

### A.1 Environment Setup and Demo

#### TL;DR:

- Setup environment as described in README.md
- Get access to [Llama-3.1-8B](#) model from HuggingFace (it is a gated model as released by Meta; access request generally gets accepted in a few minutes). More detailed instructions can be found in README.md
- `python demo.py`

The project requires a Python environment with dependencies aligned to GPU infrastructure. A Conda environment with Python 3.9 is used, along with PyTorch configured for CUDA 11.7 to match V100 cluster drivers. Additional dependencies are installed via a requirements file, and a spaCy language model is required for data preprocessing.

For data access, a setup script downloads and extracts all required datasets, including auto-labeled training data, HELM continuations, hidden-state features, and pretrained checkpoints. This ensures a consistent directory structure across runs.

A demo script is provided to showcase the full hallucination detection pipeline on a single input paragraph. It generates a model continuation, extracts hidden-state features according to a specified strategy, and outputs a hallucination prediction using a pretrained classifier. While the demo can run on a CPU, using a GPU with at least 16 GB of VRAM is strongly recommended. Inference with large language models such as LLaMA 3.1 8B or GPT-J 7B is otherwise significantly slower and may be impractical for interactive use.

Please refer to the README.md file included in the submitted repository zip for detailed environment setup instructions, including the exact bash commands.

### A.2 Running the Pipelines

The pipeline consists of sequential stages that transform raw text into features suitable for hallucination detection. It begins with auto-labeling, where hallucinated and factual sentence pairs are generated from Wikipedia using a target language model.

Next, hidden-state feature extraction is performed. Multiple strategies are supported, ranging from the original two-channel representation to richer multi-layer feature constructions that include last-token embeddings, mean-pooled representations, and inter-layer deltas. These features capture both semantic content and internal model dynamics.

For evaluation on open-domain QA, the HELM pipeline is used. Model continuations are first generated for HELM prompts, then labeled for hallucinations using OpenAI API. Hidden-state features are extracted for these continuations in the same format as training data. This ensures consistency between training and evaluation distributions.

Please refer to the README.md file included in the submitted repository zip for detailed environment setup instructions, including the exact bash commands.

### A.3 Training and Evaluating

A lightweight MLP classifier is trained on the extracted hidden-state features to distinguish between hallucinated and correct outputs. The input dimensionality is dynamically determined based on the chosen feature extraction strategy and underlying model configuration. The best-performing checkpoint is saved for downstream evaluation.

Evaluation is conducted on HELM data using both sentence-level and passage-level metrics. The system computes AUC scores for hallucination detection as well as correlation-based metrics to assess alignment with human-labeled annotations.

Results are stored as structured CSV files and can be aggregated into tables for comparison across different feature strategies. This setup enables systematic analysis of how representation choices impact hallucination detection performance.

Please refer to the README.md file included in the submitted repository zip for detailed environment setup instructions, including the exact bash commands.

### A.3 System Prompt for LLM-as-a-Judge Paradigm

The following prompt defines the evaluation protocol used in the LLM-as-judge setup for HELM benchmark extension for the newer models experimented as part of this project. It specifies the criteria under which a generated sentence is classified as hallucinated or non-hallucinated, based on its factual consistency with the provided context and real-world knowledge. The prompt is designed to enforce strict, structured outputs to ensure consistency and reliability in the generated pseudo ground truth labels.

*" You are a fact-checking judge. You will be provided with a Wikipedia-style context and a single sentence generated by a language model as a continuation. Your task is to*

*determine whether the generated sentence contains a hallucination, defined as a factual claim that is incorrect, fabricated, or not supported by reliable real-world knowledge about the subject described in the context. If the sentence contains any hallucinated factual claim, assign label = 1. If the sentence is factually consistent, trivially true, or does not contain any verifiable factual claims, assign label = 0. Return your response in strict JSON format only: {"label": 0 or 1}. “*

### **A.3 Materials Submitted:**

1. **Presentation** (Both in PPT/PDF Formats) - The one we used during the class presentation.
2. **[demo.py](#)** - This file is a demo of our project. It has 3 examples. Execute using `python demo.py`.
3. **Screen Recording** for **[demo.py](#)** execution - We made a short demonstration talking about the instructions on how this demo could be executed and the results we got.
4. **Entire Code Base** - Consists of all the python files/bash scripts we used as part of developing this project.
5. **Detailed [README.md](#)** - There are different stages as part of using our code base. Please follow the steps mentioned in this file to run it successfully.